

Technical Design

NestUp

Architecture, data model, business logic, and how it is built

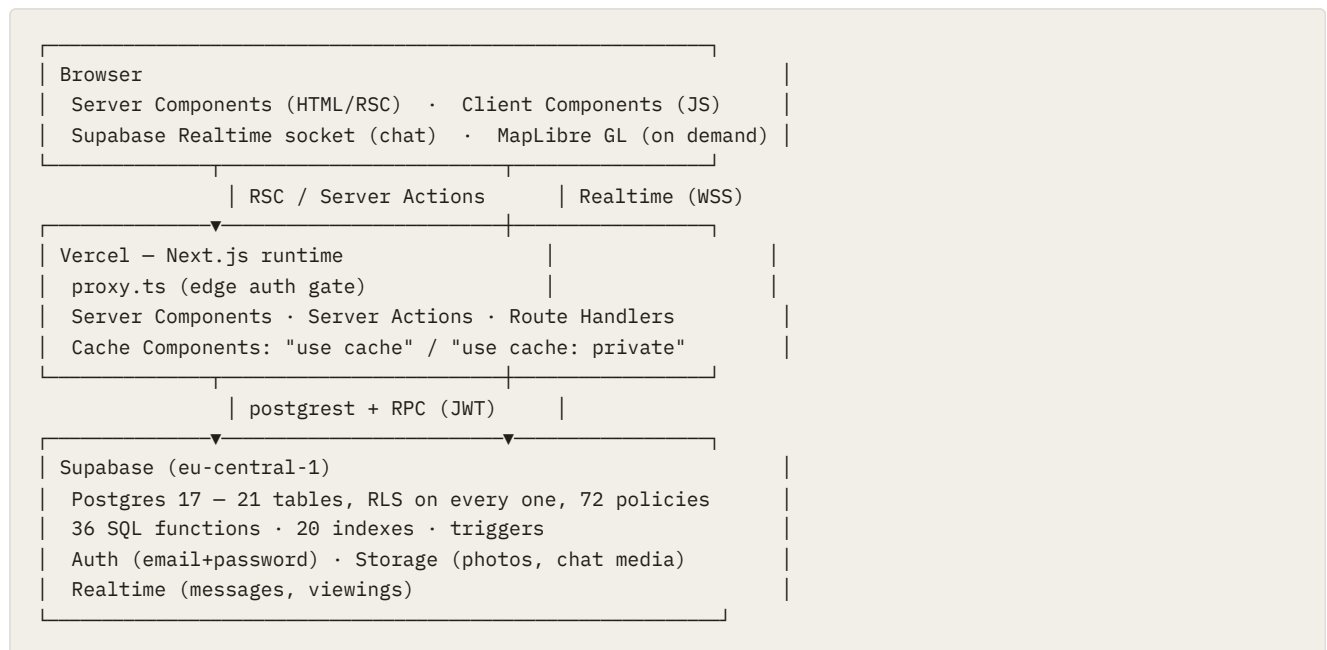
Internet Technologies — Become a Full-Stack Engineer

RUNI CS 2026 · Final project

Live product nestup-kappa.vercel.app

Repository github.com/nestupweb/nestup

1. Architecture overview



The central architectural decision: the browser never queries the database as a trusted client. Every read goes through a Server Component and every write through a Server Action, and *underneath both*, Postgres Row Level Security decides what the caller may actually see or change. Authorisation is enforced in the database, not in the application – so a bug in a page or an action cannot leak another member's data.

Why this stack

CHOICE	REASON
Next.js App Router	Server Components let the data-heavy pages (a 20-room list, a ranked deck) render on the server and ship HTML instead of JSON + a client-side fetch. Server Actions remove the need to hand-write an API layer for every mutation.
TypeScript	The domain has many near-identical shapes (<code>Profile</code> , <code>Listing</code> , <code>ConversationSummary</code>). Types catch the mix-ups the compiler can see. Required by the assignment.
Supabase	Postgres with RLS is the reason. It gives a real relational database <i>and</i> an authorisation model that lives next to the data. Auth, Storage and Realtime being in the same product removes three integrations.
Zod	One schema per input shape, used for both validation and the derived TypeScript type, so the two cannot drift.
Tailwind v4	Semantic design tokens (<code>bg-paper</code> , <code>text-ink</code> , <code>border-hairline</code> , <code>accent</code>) rather than raw palette colours, which is what makes the two themes possible without a second stylesheet.
MapLibre + CARTO	Keyless and self-hosted worker — no map vendor API key to leak, and no third-party CDN dependency at runtime.

2. Folder structure

```
Final-Project/
├─ app/                                # Next.js App Router
│  ├─ (public)/                        # No session required
│  │  └─ page.tsx                     # / → redirects to /browse
│  │  └─ browse/                      # Listings index, room page, "message the household"
│  └─ (app)/                          # Signed-in area (gated at the edge by proxy.ts)
│     ├─ swipe/                       # Personalised deck
│     ├─ chat/                       # Inbox layout + thread
│     ├─ profile/                    # Profile + edit
│     ├─ people/[id]/                # Another member's profile
│     ├─ listing/                    # Create / edit a room
│     ├─ settings/                   # Account, privacy, notifications, danger zone
│     └─ loading.tsx                 # Route-group skeleton
│  └─ (auth)/                        # Login, signup, verify, forgot/reset password
│  └─ actions/                       # 14 Server Action modules – every mutation
│  └─ api/                           # Route Handlers (JSON, or non-action side effects)
│  └─ auth/                          # confirm (email links) · signout (303 → full reload)
│  └─ layout.tsx                     # Root layout: fonts, theme, nav, view transitions
├─ components/                       # 99 components, grouped by feature
│  ├─ auth/ (8) chat/ (10) listings/ (15) map/ (11)
│  └─ profile/ (24) settings/ (8) swipe/ (4) ui/ (19)
├─ lib/                              # 53 modules – domain logic, no JSX
│  ├─ supabase/                     # 4 clients: server, browser, public (anon), middleware
│  ├─ validation/                   # 6 Zod schemas
│  ├─ compatibility.ts              # The scoring function
│  ├─ swipe.ts / swipe-deck.ts      # Deck construction (client-safe / server-only)
│  ├─ cache-tags.ts                 # Every cache tag in one place
│  └─ auth.ts                       # Session helpers + cached readers
├─ supabase/
│  ├─ migrations/                   # 43 SQL migrations, all applied to production
│  └─ templates/                    # Auth email templates (versioned, not clicked into a UI)
├─ tests/unit/                      # 97 files, 740 tests
├─ scripts/                         # Seeding, geocoding, real-browser checks
└─ docs/submission/                 # These documents
```

The organising rule: `app/` decides *what a URL renders*, `components/` is presentation, `lib/` is domain logic that could be tested without a browser. Anything that touches `cookies()` is `server-only` so it cannot be pulled into a client bundle by accident.

Route groups, and why there are three

`(public)`, `(app)` and `(auth)` do not appear in URLs — they exist so each set of pages can have its own layout and its own access assumption. `(app)` is gated at the edge; `(public)` renders for anyone and only reveals session-specific bits behind a `<Suspense>`; `(auth)` has no navigation chrome at all.

3. Database design

21 tables, **RLS enabled on every one**, 72 policies, 36 SQL functions, 20 indexes.

Core entities

TABLE	ROWS (PROD)	PURPOSE
profiles	845	One per member. Identity + how they live + what they want in roommates. PK is <code>auth.users.id</code> .
profile_details	8	Private extras — phone, contact email, social links, intro template, visibility flags. Separate table because its RLS is stricter than <code>profiles</code> .
listings	824	A room in a shared apartment. Owned by one member; co-owned via <code>listing_residents</code> .
listing_residents	1,290	Who lives in a listing's household. Turns a listing from one person's ad into a household's.
listing_invites	0	Co-poster invitations. No insert/update/delete policy — written only by two SECURITY DEFINER functions.
conversations	29	One thread per (listing, seeker). Unique constraint on the pair.
messages	57	Chat messages, with <code>client_id</code> for idempotent retry.
conversation_reads	29	Per-member read cursor — powers unread counts.
conversation_deletes	12	Per-member "delete chat" cutoff. WhatsApp semantics: hides history for one side only.
viewings	17	Apartment viewings proposed and approved inside a chat.
swipes	117	Like/skip decisions. Unique on (seeker, listing) so a room never returns.
saved_listings	33	Hearts / "Liked".
listing_views	883	History — what a member opened.
listing_dwell	24	How long a card was looked at — an engagement signal.
blocks	2	Mutual block. Removes both directions from decks and search.
reports	0	Reported members and listings.
suspensions	0	No write policy at all — written only by <code>apply_report_suspension()</code> . A member cannot lift their own suspension.
google_tokens	0	OAuth tokens for calendar export. Owner-only.
app_config	1	Server-side knobs. No RLS policies; SECURITY DEFINER functions only.
auth_mail_throttle	10	Rate-limits outgoing auth email.
matches	0	Reserved for mutual-match semantics; superseded by conversations.

Design decisions worth defending

profiles.user_id is the primary key and a foreign key to **auth.users**. There is no separate profile id. This makes every RLS policy in the schema a comparison against `auth.uid()` with no join, and makes account deletion cascade correctly by construction.

Two tables for one person (profiles / profile_details). `profiles` is readable by other members — it is what the deck and the room page render. `profile_details` holds the phone number and contact email and is owner-only, with a `public_profile_details()` function that returns only the fields the owner chose to expose. Splitting them means the

default is private: a new column added to `profile_details` is not accidentally world-readable.

Denormalised `household_size` and `household_gender` on `listings`. Maintained by triggers over `listing_residents`. They exist because the browse filter and the deck both need them on every row, and computing them per row would turn one query into 800.

Soft deletion, not hard. `listings.removed_at`, `listings.taken_at`, `conversation_deletes.cleared_at`. A room that is gone must not take its conversations with it — people who talked about a room still have that history. This is why there are two SELECT policies on `listings`: one for the public list, one for "I am linked to this listing through a conversation".

`messages.client_id`. The browser generates a UUID before sending. A retry of the same id hits a unique index and returns the existing row instead of double-posting. This is what makes the optimistic UI safe on a flaky connection.

Indexes

20 indexes, placed where the query planner would otherwise scan: `swipes(seeker_id, listing_id)`, `messages(conversation_id, created_at)`, `listings(is_active, city)`, `listing_views(user_id, viewed_at)`, `saved_listings(user_id)`, `blocks(blocker_id)`, `viewings(conversation_id)` and others. Discussed further in the [Scale](#) document.

4. Key CRUD operations

Every mutation is a **Server Action**. There are no client-side database writes anywhere in the product.

ENTITY	CREATE	READ	UPDATE	DELETE
Profile	<code>saveProfileAction</code> (first save = onboarding)	<code>getCachedOwnProfile</code> , <code>/people/[id]</code>	<code>saveProfileAction</code> , <code>saveAboutAction</code>	<code>deleteAccountAction</code> (RPC <code>delete_own_account</code>)
Listing	<code>saveListingAction</code>	<code>queryListings</code> , <code>getListing</code>	<code>saveListingAction</code>	<code>removeListingAction</code> (soft), <code>setListingActiveAction</code> (pause), <code>markTakenAction</code>
Photos	<code>checkAndUploadPhotoAction</code> (AI-checked, then Storage)	Public bucket URL	Reorder in form	Remove in form
Swipe	<code>recordSwipeAction</code>	<code>getCachedDeck</code>	Upsert on conflict	— (a decision is permanent)
Saved	<code>setSavedAction</code>	<code>getSavedListingIds</code>	—	<code>setSavedAction(false)</code>
Conversation	<code>findOrCreateConversation</code>	<code>getCachedInbox</code> , thread page	<code>markReadAction</code>	<code>deleteConversationAction</code> (per-member cutoff)
Message	<code>sendMessageAction</code>	Thread page + Realtime	—	—
Viewing	<code>proposeViewingAction</code>	Thread page	<code>respondViewingAction</code>	Cancel via status
Invite	<code>inviteRoommates</code> (RPC)	<code>getPendingInvites</code>	<code>respondToInviteAction</code> (RPC)	—
Moderation	<code>reportAction</code> , <code>blockUserAction</code>	<code>getBlocked</code>	—	<code>unlockUserAction</code>

Why Server Actions rather than REST for mutations: the action is a typed function the component imports. There is no URL to keep in sync, no request/response shape to hand-write, and no way to call it without the framework attaching the session. The four Route Handlers that *do* exist are the cases where an action does not fit.

5. API surface

Route Handlers (`app/api/` , `app/auth/`)

ROUTE	METHOD	WHY IT IS A HANDLER AND NOT AN ACTION
<code>/api/listings</code>	GET	Queried by client-side code; needs a JSON response
<code>/api/listings/pins</code>	GET	All placed rooms for the map — ~150 KB, fetched only when the map is opened
<code>/api/listings/[id]/invites</code>	GET	Co-poster state for the listing form
<code>/api/invites</code> , <code>/api/invites/[id]</code>	GET/POST	Invitation list and response
<code>/api/places</code>	GET	Proxies Overpass for cafés/shops near a room — keeps the upstream call server-side
<code>/api/google/connect</code> , <code>/api/google/callback</code>	GET	OAuth redirect flow; a browser must be redirected here
<code>/auth/confirm</code>	GET	Where every emailed auth link lands
<code>/auth/signout</code>	POST	Must answer 303 so the browser performs a full document load and drops all client caches

API routes check authorisation inside the handler. The edge proxy only guards page routes — an API caller must receive a JSON 401, not an HTML redirect to `/login` .

SQL functions (RPC)

36 functions. The important ones are `SECURITY DEFINER` — they run with elevated rights precisely so the *table* can have no write policy at all:

- `my_conversations()` , `my_unread_count()` — inbox and badge, RLS-scoped internally
- `mark_conversation_read()` , `clear_conversation()` — read cursor and per-member delete
- `delete_own_account(p_heir)` — deletes the account and **hands a shared listing to a roommate in the same transaction**
- `invite_listing_roommates()` , `respond_to_listing_invite()` — the only writers to `listing_invites`
- `apply_report_suspension()` — the only writer to `suspensions`
- `public_profile_details()` — returns only the fields a member chose to expose
- `blocked_user_ids()` , `is_blocked()` , `is_suspended()` , `linked_to_listing()` — policy helpers

This is the pattern worth explaining: **when a table must only ever be written one specific way, give it no write policy and one definer function.** There is then no path to a wrong write, rather than a policy that tries to describe every wrong write.

6. Core business logic

6.1 Compatibility scoring (`lib/compatibility.ts`)

Two independent scores, both 0–100.

Lifestyle — weighted sum, weights summing to 100:

COMPONENT	WEIGHT	COMPONENT	WEIGHT
Budget	20	Sleep schedule	6
City	18	Guests	6
Move-in date	10	Noise	4
Smoking	10	Diet	4
Cleanliness	10	Shabbat	4
Pets	8		

Social — overlap of declared interests.

Two conventions carry most of the design:

1. **A missing preference scores ~60% of its weight, not zero.** "I did not say" is not the same as "we disagree". Scoring it as a mismatch would punish incomplete profiles and make the deck reward form-filling rather than fit.
2. **Scoring is directional.** Perspective is "seeker" or "lister", because each side judges *their* preferences against *the other's* reality. A seeker who tolerates guests and a household that has them often is a good match from the seeker's side; the reverse question is a different one.

6.2 The deck (`lib/swipe.ts` , `lib/swipe-deck.ts`)

1. **Hard filters** — the room's city must be one of the seeker's preferred cities; exclude rooms already swiped, own rooms, paused/removed/taken rooms, and anything belonging to a blocked member.
2. **Score** both dimensions.
3. **Gate:** `sortBy(lifestyle, social) < MIN_DECK_SCORE (60)` → drop it entirely.
4. **Rank** by `sortBy` and cap at `DECK_SIZE (60)`.

The gate is the product decision. It is why the deck can legitimately be empty, and why a "no strong matches yet" state had to be designed rather than treated as an error.

The split across two modules is a real constraint, not tidiness: `lib/swipe.ts` is imported by client components, so it must be safe in a browser bundle; `lib/swipe-deck.ts` is `server-only` and holds the cookie-reading Supabase client.

6.3 Household semantics

- One person, one home — a unique index gives an owner at most one active listing.
- A listing's owner cannot be reassigned by a normal `UPDATE` (a trigger refuses it); only the account-deletion handover, which opens a transaction-local GUC, may do it.
- Deleting an account hands a co-owned listing to an eligible roommate instead of cascading it away.

7. State management

There is **no state management library** — no Redux, no Zustand, no React Query. This is a deliberate choice, and the reasoning is the clearest example of the App Router's model paying off:

KIND OF STATE	WHERE IT LIVES
Server data	Server Components. It is not client state at all; it is rendered output.
Cached server data	Next.js Cache Components — <code>"use cache"</code> (shared) and <code>"use cache: private"</code> (per-browser), invalidated by tag.
Form state	<code>useActionState</code> via a <code>useStickyForm</code> wrapper, so a rejected form keeps what the user typed.
Ephemeral UI state	<code>useState</code> — open dialogs, the current swipe card, filter drafts.
URL state	<code>searchParams</code> — filters, sort, page, active tab. Shareable and back-button correct by construction.
Realtime	A Supabase subscription that invalidates a cache tag; the server re-renders.

Adding a client cache library would have meant maintaining a second copy of the server's data with its own invalidation rules. The cache-tag system already does that job, in one place (`lib/cache-tags.ts`), with the server as the single source of truth.

The caching layer

Five cached reads, each with a tag and a lifetime:

READ	CACHE	TAG
<code>queryListings</code>	shared	<code>listings</code>
<code>getSavedListingIds</code>	private	<code>saved:<userId></code>
<code>getProfileTabData</code>	private	<code>profile:<userId></code>
<code>getCachedDeck</code>	private	<code>deck:<userId></code>
<code>getCachedInbox</code>	private	<code>chat:<userId></code>

`"use cache: private"` results live **in the requesting browser's memory only** and are never written to a shared server store — which is what makes it safe to cache one member's deck and inbox. Every mutation calls `updateTag` for exactly what it changed, so hearting a room does not throw away the member's chats.

8. Error handling

Server Actions return errors; they do not throw. Every action returns a discriminated union (`{ ok: true, ... } | { ok: false, error: string }` or `{ error?: string }`), so the form can render the message next to the field. A thrown error would produce an error boundary — the right response to a bug, the wrong response to a bad password.

Layered:

1. **Database constraints and RLS** — the last line. A refused write returns an error code the action translates. `delete_own_account` raises `pick_heir / bad_heir` hints, which the action turns into "Choose which roommate takes over your listing."
2. **Server Actions** — validate with Zod, return field-level messages, never leak internals. `sendMessageAction` treats Postgres `23505` (unique violation) as *success* — that id was already delivered.
3. **Route Handlers** — JSON with a proper status. 401 for unauthenticated API callers.
4. **React error boundaries** — `app/(public)/browse/error.tsx`.
5. **Deliberate non-failures** — `getUnreadCount()` catches everything and returns 0. It is a decoration on a badge, handed to the layout unawaited; an unhandled rejection there would take down a whole page for a number.

Messages say what to do next. "That code is wrong or has expired. Check the email, or send a new one" beats "Invalid OTP".

9. Input validation

Every input is validated twice, in two different places, on purpose.

1. **Client** — HTML constraints and immediate feedback. Convenience only; assumed bypassable.
2. **Server (Zod)** — six schemas in `lib/validation/`: `profile`, `listing`, `message`, `filters`, `about`, `report`. The Server Action parses before touching the database. The TypeScript type is *inferred from the schema*, so validation and types cannot drift.
3. **Database** — `CHECK` constraints (`age` between 18 and 120, `bio` ≤ 500 , `cleanliness` between 1 and 5, `budget_max = 0` or `budget_max >= budget_min`), enums, foreign keys, unique indexes.

Specific hardening worth naming:

- **URL parameters are parsed, not trusted.** `listingFiltersSchema` uses `.catch()` throughout, so a hand-edited `?safe_room=nonsense` falls back to "unset" rather than erroring.
 - **Open redirects.** `sanitizeNextPath()` guards every `?next=` — only same-site paths survive.
 - **File uploads.** Type and size checked client-side, re-checked server-side; a chat photo must live under its own conversation's storage prefix.
 - **Photo content.** `checkAndUploadPhotoAction` sends the bytes to Gemini and stores the file **only if** the image matches its declared tag — a bedroom slot cannot hold a picture of a car.
-

10. UX design

Principles

- **Mobile-first.** Most people look for a room on a phone. A floating pill bottom-nav; the nav hides while a chat thread is open so the composer owns the bottom edge.
- **Two themes.** Editorial (light) and Noir (dark, accent turns gold), driven entirely by semantic tokens.
- **One typeface** — Inter, everywhere.
- **Nothing renders as "empty by accident".** Every empty state says what happened and what to do — an empty deck is a strict filter, not a broken page.

Navigation

Four destinations: **Swipe · Listings · Chat · Profile**. Tab-to-tab moves slide left or right by tab order using the View Transitions API; anything else crossfades.

Perceived performance

The pages are structured so the static frame ships first and only the member-specific part streams:

- Layouts read no session, so the shell is in the prerender.
- Data-heavy sections sit behind `<Suspense>` with skeletons shaped like the real content, so the swap is a fill rather than a jump.
- Cached reads ride along in the prefetched App Shell, so returning to Swipe, Chat or Profile is **~60 ms with no skeleton at all** (measured on production).

Deliberate restraint

No map is drawn until an icon is pressed. MapLibre is a large dependency and most visits never open a map; loading it eagerly would tax every visit for a feature used on some of them.
